Reviewer Pack — Frobenius-Schur Indicator and Quantum Circuit Entropy

Entry df6a40ce8d52 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 20:04:53 UTC

Frobenius-Schur Indicator and Quantum Circuit Entropy

Entry ID: `df6a40ce8d52` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 20:04:53 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Lie Groups **Field B** (complexity object): Quantum Circuit Complexity

Statement:

> The Frobenius-Schur indicator of the characteristic polynomial of a quantum circuit's unitary transformation is upper bounded by the entropy of its output distribution, i.e., |Frobenius-Schur Indicator(U)| ≤ Entropy(D), where U is the unitary matrix representing the quantum circuit and D is its output distribution.

Rationale (proposer's reasoning):

> The representation theory of Lie groups provides a framework to study the properties of linear transformations. The Frobenius-Schur indicator captures non-trivial information about the structure of these transformations. Quantum circuits, being essentially compositions of unitary matrices, exhibit complex behavior that can be analyzed through their output distributions. This conjecture bridges these two fields by proposing a quantitative relationship between a mathematical invariant from representation theory and an entropy measure from quantum information theory.

Taxonomy category: `LieGroupRepresentation_QuantumCircuitComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e43fabe022f9d199`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 independent random seed generations, the mean of |Frobenius-Schur Indicator(U)| values does not exceed the corresponding Entropy(D) values for all generated quantum circuits.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius-Schur indicator" AND "quantum circuit complexity" - "characteristic polynomial" AND "output distribution entropy" - "Lie group representation theory" AND "unitary transformation entropy"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def determinant(A):
    n = len(A)
    det = Fraction(1)
    for i in range(n):
        det *= A[i][i]
    return det

def trace(matrix):
    n = len(matrix)
    tr = 0
```

```
    for i in range(n):
        tr += matrix[i][i]
    return tr

def unitary_matrix(n):
    U = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):
        U[i][i] = Fraction(1, 1)
    return U

def frobenius_schur_indicator(U, n):
    n = len(U)
    identity = unitary_matrix(n)
    result = trace(matrix_multiplication(identity, U.conjugate().transpose())) / math.factorial(n)
    return abs(result)

def matrix_multiplication(A, B):
    n = len(A)
    C = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def entropy(distribution):
    H = Fraction(0, 1)
    for p in distribution:
        if p > 0:
            H -= p * math.log2(p)
    return H

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    U = unitary_matrix(n)
    distribution = [random.random() for _ in range(2**n)]
    total = sum(distribution)
    distribution = [p / total for p in distribution]

    metric_value = frobenius_schur_indicator(U, n)
    entropy_value = entropy(distribution)

    conjecture_holds = abs(metric_value) <= entropy_value
    counterexample = "" if conjecture_holds else "Frobenius-Schur Indicator > Entropy"

    return {
        "metric_name": "Frobenius-Schur Indicator",
        "metric_value": metric_value,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Frobenius-Schur Indicator > Entropy\" first_fai
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9b9aa18.py", line 106, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9b9aa18.py", line 85, in run_trial
    metric_value = frobenius_schur_indicator(U, n)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f9b9aa18.py", line 57, in frobenius_sch
    result = trace(matrix_multiplication(identity, U.conjugate().transpose())) / math.factorial(n)
                                                   ^^^^^^^^^^^
AttributeError: 'list' object has no attribute 'conjugate'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed due to an AttributeError, which prevented the computation of the Frobenius-Schur Indicator and Entropy values necessary to evaluate the conjecture. |

next: Investigate the cause of the AttributeError and correct the code before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14879
2	propose	ollama_remote	glm4:latest	0	0	15591
3	preregistration	ollama_remote	glm4:latest	0	0	17212
4	novelty	ollama_remote	glm4:latest	0	0	8279
5	novelty	ollama_remote	glm4:latest	0	0	12225
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15364
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10316
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40459
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11568
10	judge	ollama_remote	glm4:latest	0	0	47230

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 193122 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/df6a40ce8d52.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/df6a40ce8d52.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/df6a40ce8d52.tar.gz` (if generated)